CS2030 (2410) Practical Assessment #1

Tags & Categories

Tags:

Categories:

Related Tutorials

Task Content

CS2030 (2410) Practical Assessment #1

Based on a true story...

After almost a month of non-stop work, Dad wanted to atone for the loss of family time by treating everyone to watch a musical performance. As it was decided late, Dad hastily tried to book tickets online; there was only a pocket of three seats at a corner in the last row. Dad quickly selected the seats, and proceeded to make payment.. but *arghhh..* the browser hung!!!

He quickly refreshed the booking site but to his horror, the seats were already booked! He tried to clear the browser cache, use incognito mode, use different browsers, different devices... all to no avail! Someone must have booked the same three seats! Dad was left exasperated.

An hour later, Mum accessed the booking site out of curiosity and found that the same seats were still available. Long story short... Mum saved the day!

But why would the booking system keep the seats booked even if payment did not proceed? There was something *very* wrong with the design of the system!

Task

Your task is to design a better seat-booking system. Users can access the system at the same time, but seats are only locked upon payment. In this way, if one user fails to book the seats, other users can still proceed. In the case of Dad, he would just need to refresh and book the seats again.

Take Note!

This task comprises a number of levels. You are required to complete ALL levels.

The following are the constraints imposed on this task. In general, you should keep to the constructs and programming discipline instilled throughout the module.

- Write each class or interface in its own file. Do not use single letter names for classes or interfaces.
- Ensure that ALL object properties and class constants are declared `private final`
- Ensure that ALL classes/interfaces are NOT cyclic dependent.
- ONLY the following java libraries ARE allowed:
    - `java.util.List`
    - `java.util.stream.IntStream`
    - `java.util.stream.Stream`

    - functional interfaces from `java.util.function`
- The following are NOT allowed:
    - `var`, `null`, `default`, `enum`
    - `instanceof` (except checking for an instance of its own class)
- There is no need to use bounded wildcards.
- You are NOT allowed to define anonymous inner classes; if necessary, define lambdas instead.
- Other usual restrictions:
    - Use only `&&`, `||` and `!` in logical expressions. DO NOT use bitwise operators.
    - You are NOT allowed to use `*` wildcard imports.
    - You are NOT allowed to use method references `::`
    - You are NOT allowed to define array constructs, e.g. `String[]` or using ellipsis, e.g.`String...`
    - You are NOT allowed to use exception handling
    - You are NOT allowed to use Java reflection, i.e. `Object::getClasses` and other methods from `java.lang.Class`

A minimal `Pair` class (not a record, but behaves similarly) has been provided for you. DO NOT replace this with your own version.

Level 1

Let's start by creating a seating plan. The `Seat` interface has been provided for you. DO NOT modify it.

```
interface Seat {
    public boolean isBooked();
}
```

Create the `Available` and `Booked` classes as implementations of `Seat` so that calling `isBooked()` returns `false` and `true` respectively. Moreover, include the `toString` method that correspondingly returns the string representation `.` and `B`.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order
jshell> Stream.<Seat>of(new Available(), new Booked()).map(x -> x.isBooked()).toList()
$.. ==> [false, true]

jshell> Stream.<Seat>of(new Available(), new Booked()).forEach(x -> System.out.println(x))
.
B
```

Level 2

We simplify the seating arrangement as one single row of seats. Seats are numbered starting from `0`, just like in regular list indexing.

Create the `Seating` class following the specifications below:

- `Seating(int capacity)`

  takes in a positive integer capacity and creates a seating plan of all available seats.

- `boolean isAvailable(Pair<Integer,Integer> rowOfSeats)`

  takes in a row of seats specified as a pair of values comprising the starting seat number and the number of consecutive seats, and proceeds to test if all seats are available. You will also need to check the avaiability (as well as validity) of the row of seats against the current seating plan.

- `Seating book(Pair<Integer,Integer> rowOfSeats)`

  books the row of seats specified. If seats are available, the updated seating plan is returned; otherwise the original seating plan is returned.

- `public String toString()`

  returns the string representation of the seating plan.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order
jshell> new Seating(20)
$.. ==> ....................

jshell> new Seating(20).isAvailable(new Pair<Integer,Integer>(1,5))
$.. ==> true

jshell> new Seating(20).book(new Pair<Integer,Integer>(1,5))
$.. ==> .BBBBB..............

jshell> new Seating(20).book(new Pair<Integer,Integer>(1,5)).
   ...> isAvailable(new Pair<Integer,Integer>(5,5))
$.. ==> false

jshell> new Seating(20).book(new Pair<Integer,Integer>(1,5)).
   ...> book(new Pair<Integer,Integer>(5,5))
$.. ==> .BBBBB..............

jshell> new Seating(20).isAvailable(new Pair<Integer,Integer>(15,10))
$.. ==> false

jshell> new Seating(20).book(new Pair<Integer,Integer>(15,10))
$.. ==> ....................

jshell> new Seating(20).isAvailable(new Pair<Integer,Integer>(-5,10))
$.. ==> false
```

Level 3

Booking and purchasing seats are handled via different transactions. We shall first look at the `Request` (request-to-book) transaction, just one of many sub-classes of `Transaction`.

When a user makes a booking request, he/she selects a row of seats from a given seating plan. As part of the purchase later, the billing number as well as the bank is required.

Create a constructor `Request` so as to instantiate a request transaction. Note that this is only a request, no actual booking or billing is done.

```
Request(Seating plan, Pair<Integer,Integer> rowOfSeats, int billing, Bank bank)
```

Here is an example when a request for five seats are made on an empty seating plan.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order
jshell> Transaction t = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(1,5), 2030, x -> x % 2 == 0)
t ==> REQUEST:
Requesting
....................
```

The first two arguments are the seating plan and the row of seats to be booked (availability will be checked later). The third argument is an integer billing identifier.

The fourth argument is interesting. `Bank` parameter takes in a lambda expression where the input is an integer and the output is a `boolean` value. Doing this allows us to simulate a successful or failed purchase when the lambda expression is applied on the billing identifier. In the above example, since the bank identifier is even, the purchase will eventually proceed. If all seems confusing, right now just figure out how to correctly define `Bank`.

The output comprises three parts:

- First line is the type of the transaction, in this case `REQUEST`
- Next line is a log of the transaction; this log will grow as the request transitions to other transactions. As of now, it is only a single line.
- Last line is just the seating plan from which the user was considering his/her booking choice.

You may assume that the seats that are booked by a user are valid. You will need to check the validity again later when the actual booking takes place.

Level 4

Suppose three users are planning to book seats at around the same time. They will all see the same seating plan. Let's assume no seats have been booked yet.

```
jshell> Transaction r1 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(10, 10), 2030, x -> x % 2 == 0)
r1 ==> REQUEST:
Requesting
.....................

jshell> Transaction r2 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(15,2), 2040, x -> false)
r2 ==> REQUEST:
Requesting
.....................

jshell> Transaction r3 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(8,2), 1010, x -> false)
r3 ==> REQUEST:
Requesting
.....................
```

All request transactions will now be processed one by one. We start off with an initial transaction `Init`. This transaction encapsulates the current seating which will be used for the actual booking.

```
jshell> Transaction init = new Init(new Seating(20).book(new Pair<Integer,Integer>(1,5)))
init ==> INIT:
Initializing
.BBBBB..............
```

The first request transaction takes in the initial transaction and attempts to perform a booking.

```
jshell> Transaction afterR1 = r1.transact(init)
afterR1 ==> APPROVED:
Initializing
billed 2030; booked 10--19
.BBBBB....BBBBBBBBBB
```

Notice that the transaction is approved because seats are available and the bank approves the billing (because `2030` is even). Billing identifier and range of seats booked are also logged. The resulting transaction is an `Approve` transaction.

The next request transaction proceeds.

```
jshell> Transaction afterR2 = r2.transact(afterR1)
afterR2 ==> APPROVED:
Initializing
billed 2030; booked 10--19
.BBBBB....BBBBBBBBBB
```

Notice that nothing happens. This is because the seats are no longer available. For simplicity, we do not need to log since no attempt to bill takes place. In this case, the transaction returned will be the same as the one after the `r1` transaction, as though `r2` has not taken place.

Now for the third request,

```
jshell> r3.transact(afterR2)
$.. ==> REJECTED:
Initializing
billed 2030; booked 10--19
not billed 1010
```

```
.BBBBB....BBBBBBBBBB
```

In this case, the seats are available, but the bank rejects the billing. This is logged in the transaction. The resulting transaction is a `Reject` transaction. Note that this only means that the last transaction is rejected, not all transactions have been rejected.

Your task is to write the `Init`, `Approve` and `Reject` transaction classes. All transactions classes require the following method to be defined.

```
Transaction transact(Transaction t)
```

Tip: for those transactions whose `transact` method is never called, you can simply return `this`.

Level 5

Now the easy part. Within the given `Main.java`, include the method

```
Transaction process(Stream<Transaction> transactions, Init init)
```

that takes in a stream of request transactions, processes them one by one using the seating plan within the `init` transaction. The method returns the resulting transaction after processing all requests.

```
$ javac --release 21 --enable-preview Main.java
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> Transaction r1 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(10, 10), 2030, x -> x % 2 == 0)
r1 ==> REQUEST:
Requesting
....................

jshell> Transaction r2 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(15,2), 2040, x -> false)
r2 ==> REQUEST:
Requesting
....................

jshell> Transaction r3 = new Request(new Seating(20),
   ...>    new Pair<Integer,Integer>(8,2), 1010, x -> false)
r3 ==> REQUEST:
Requesting
....................

jshell> Init init = new Init(new Seating(20).book(new Pair<Integer,Integer>(1,5)))
init ==> INIT:
Initializing
.BBBBB..............

jshell> process(Stream.of(r1, r2, r3), init)
$.. ==> REJECTED:
Initializing
billed 2030; booked 10--19
not billed 1010
.BBBBB....BBBBBBBBBB
```

Submission (Practice)

Your Files:

BROWSE

SUBMIT (only .java, .c, .cpp, .h, .jsh, and .py extensions allowed)

To submit multiple files, click on the Browse button, then select one or more files. The selected file(s) will be added to the upload queue. You can repeat this step to add more files. Check that you have all the files needed for your submission. Then click on the Submit button to upload your submission.